



**STANDARD**

**Floating Point to Integer Mapping**

**24 June 2021**

## 1 Scope

This standard describes the method for mapping floating-point values to integer values and the reverse, mapping integer values back to their original floating-point value to within an acceptable precision. There are many ways of optimizing the transmission of floating-point values from one system to another; the purpose of this standard is to provide a single method for use by all MISB metadata standards. This standard supports all floating-point ranges and valid precisions. This standard provides a method for a forward and reverse linear mapping of a specified range of floating-point values to a specified integer range of values based on the number of bytes desired for the integer value. Additionally, it provides a set of special values to transmit non-numerical “signals” to a receiving system. This standard is dependent on context from an invoking standard (or another document), called a Parent Document.

## 2 References

- [1] IEEE 754-2008 Standard for Floating-Point Arithmetic [and Floating-Point formats], 2008.
- [2] MISB ST 0601.17 UAS Datalink Local Set, 29 Oct 2020.

## 3 Acronyms and Definitions

|              |                                                            |
|--------------|------------------------------------------------------------|
| <b>IEEE</b>  | Institute of Electrical and Electronics Engineers          |
| <b>IMAPA</b> | [Floating Point to] Integer Mapping using Starting Point A |
| <b>IMAPB</b> | [Floating Point to] Integer Mapping using Starting Point B |
| <b>KLV</b>   | Key Length Value                                           |
| <b>MISB</b>  | Motion Imagery Standards Board                             |
| <b>msb</b>   | Most Significant Bit                                       |
| <b>ST</b>    | Standard                                                   |

**ceiling**  
[ $x$ ] Rounding any non-integer value up toward  $+\infty$ . The ceiling function notation is [ $x$ ] and the definition of the ceiling function is: [ $x$ ] =  $\min\{n \in \mathbb{Z} | n \geq x\}$ . Examples:  $[-1.1] = -1$ ;  $[1.1] = 2$ . Note: Microsoft Excel (pre-2010) does not perform this operation correctly.

**floor**  
[ $x$ ] Rounding any non-integer value down towards  $-\infty$ . The floor function notation is [ $x$ ] and the definition of the floor function is: [ $x$ ] =  $\max\{m \in \mathbb{Z} | m \leq x\}$ .

|                  |                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                  | Examples: $\lfloor -1.1 \rfloor = -2$ ; $\lceil 1.1 \rceil = 1$ . <u>Note: Microsoft Excel (pre-2010) does not perform this operation correctly.</u>                                                                                                                                                                                                                                                                                   |
| <b>integer</b>   | An integer is a whole number without a fractional value or a real value with a fractional value set to zero. Integers are represented using the standard integer set notation, $\mathbb{Z}$ .                                                                                                                                                                                                                                          |
| <b>truncate</b>  | A rounding function which removes the fractional part of a real number. The truncate function notation is $\text{truncate}(x)$ , where $x \in \mathbb{R}$ .<br>Example: $\text{truncate}(100.2) = 100$ .                                                                                                                                                                                                                               |
| <b>real</b>      | In this standard the definition of a real is simplified to be, “a whole number and a fractional value”. The fractional value may be zero which means the value is also an integer. Reals are represented using the standard reals set notation, $\mathbb{R}$                                                                                                                                                                           |
| <b>round</b>     | A rounding function which, for this standard, means $\lfloor x + 0.5 \rfloor$ if $x \geq 0$ and $\lceil x - 0.5 \rceil$ if $x < 0$ . <u>Note the floor and ceiling notation in the definition.</u> Examples: $\text{round}(-1.1) = \lfloor -1.1 - 0.5 \rfloor = -2$ ; $\text{round}(-1.6) = \lfloor -1.6 - 0.5 \rfloor = -2$ ; $\text{round}(1.1) = \lceil 1.1 + 0.5 \rceil = 2$ ; $\text{round}(1.6) = \lceil 1.6 + 0.5 \rceil = 2$ . |
| <b>rounding</b>  | Changing the fractional part of a real value to zero by some technique (e.g., round, ceiling, floor), i.e., forcing the fractional part of a real number to be zero.                                                                                                                                                                                                                                                                   |
| <b>precision</b> | (From [1]) The maximum number, $p$ , of significant digits in a numerical format, or the number of digits a result is rounded to. <u>Note: This is not the same definition that refers to repeatability of measurements as used in other MISB documents and references.</u>                                                                                                                                                            |

## 4 Revision History

| Revision  | Date       | Summary of Changes                                                                                                                                                                                                                                                   |
|-----------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ST 1201.5 | 06/24/2021 | <ul style="list-style-type: none"> <li>Added special values when mapping value exceeds the minimum or maximum bound</li> <li>Added Req -18</li> <li>Added definition of reverse IMAPA and IMAPB (i.e., RIMAPA, RIMAPB)</li> <li>Added example test values</li> </ul> |
|           | 11/06/2021 | <ul style="list-style-type: none"> <li>Editorial clarifications on the implementation and use of IMAP_BELOW_MINIMUM and IMAP_BELOW_MAXIMUM</li> <li>Adjusted examples to reflect editorial changes</li> </ul>                                                        |
|           | 05/15/2022 | <ul style="list-style-type: none"> <li>Adjusted the “Reserved” section of the Special Value Bit Patterns Table to contain all Reserved bit patterns</li> </ul>                                                                                                       |

## 5 Introduction

Systems transmit and receive measured and computed floating-point values between systems. Oftentimes, the values do not fully utilize the floating-point range or precision afforded, and thus provide an opportunity to reduce the number of bytes representing the data. Additionally, systems need to communicate non-numeric special values or “signals”; for example, a sensor value indicating “beyond measurement range” or in cases where a divide by zero occurs (i.e.,

+infinity). This standard applies to IEEE 754 [1] floating-point values represented in 16, 32, 64, and 128-bit precision, and includes IEEE special values for infinity, and NaN's (Not-a-Number).

For convenience, Section 7 succinctly states the algorithm for developers to implement and Section 8 informatively shows the derivation of the algorithm and format.

## 6 Algorithm Principles

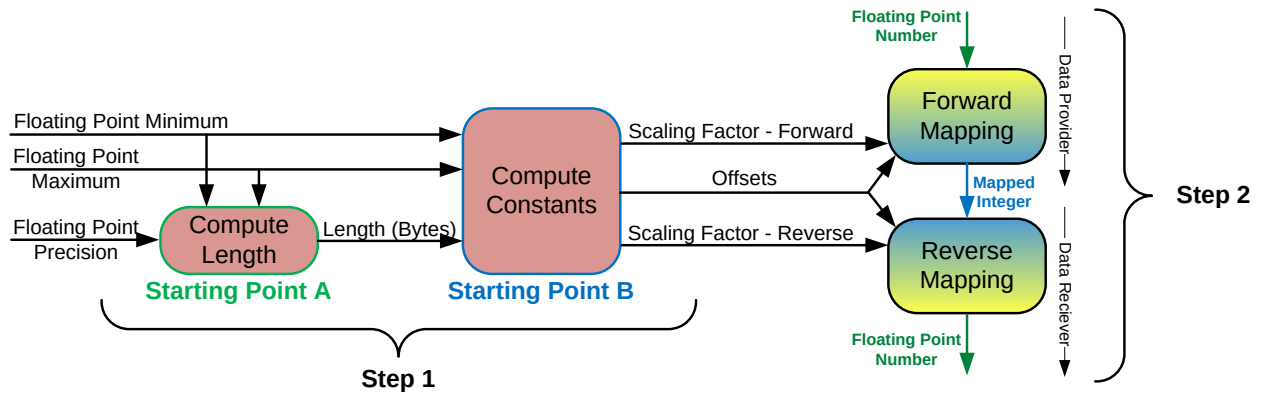
The following principles underlie the IMAP algorithm:

- The binary representation of a floating-point value is an IEEE 754-2008 binary interchange format floating-point value.
- The algorithm performs the mapping based on a [user] specified floating-point range from a min value to a max value – inclusive (including the end points).
- The algorithm is a linear mapping from a floating-point value to a non-negative integer value of [user] defined length in bytes.
- The binary representation of a non-negative integer value is in the format of a standard unsigned integer, with the length, in bytes, defined by the [user's] implementing Parent Document.
- The algorithm provides an inverse mapping from a non-negative integer value to the original floating-point value within a [user] specified precision. (Note: the mapping algorithm can supply better precision than what the user specifies. The defined length and precision are related – given one the other can be computed but both cannot be specified by the user).
- If the [users] floating-point range includes 0.0, then the algorithm maps 0.0 exactly to a specific integer value, called a “zero offset.”
- All negative floating-point values map to an integer that is less than the zero offset.
- All positive floating-point values map to values greater than or equal to the zero offset.
- The resulting format provides a set of special values which map to the IEEE 754-2008 special values, including:  $\pm$  infinity,  $\pm$ QNaN and  $\pm$ SNaN (note: negative zero is not included in this requirement).
- The resulting format provides a bit space for custom signals to be sent by the user as defined by the [users] implementing Parent Document (e.g., MISB ST 0601).
- The algorithm is designed for fewest operations as possible (during the actual mapping) for computational efficiency.

## 7 Mapping Algorithm and Integer Format

This section describes the algorithm for mapping the floating-point values to integer values, and the reverse mapping of integers back to floating-point values. Additionally, this section defines the notation to use in any MISB Parent Document when “invoking” this standard. Section 8 is an informative section which describes the development of this algorithm in detail. The algorithm is based on two steps as shown in Figure 1:

## ST 1201.5 Floating Point to Integer Mapping



**Figure 1: Functional view of the two-step conversion process**

**Step 1:** The first step computes one-time use constants, and the second step is the forward or reverse mapping. There are two “starting points” for the first step. With known precision, minimum, and maximum values Starting Point A computes the mapped value length for use in Starting Point B. With known length, minimum, and maximum values Starting Point B computes the Scaling Factor Constants for the forward and reverse mapping step (Step 2). Starting Point A equates to IMAPA and Starting Point B equates to IMAPB as discussed in Section 7.2.3.1.

**Step 2:** The second step uses the constants computed from Step 1 to perform floating-point-to-integer and/or integer-to-floating-point mappings – provided the floating-point minimum, maximum, and length do not change. Figure 1 illustrates the interaction of the two steps.

Steps 1 and 2, described in the sections below, use the standard order of operations (i.e., operations in parenthesis first, exponents/roots second, multiplication/division third, addition/subtraction fourth).

### 7.1 Step 1: Scaling Factors & Offsets

Step 1 has two starting points based on whether the floating-point precision is known or the length (in bytes) of the resulting mapped integer is known.

#### 7.1.1 Starting Point A: Specifying Precision

| Input                                 |                                                                                                                 |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| •                                     | a = Floating-Point Minimum, range is from smallest float value to largest float value                           |
| •                                     | b = Floating-Point Maximum, range is from smallest float value to largest float value                           |
| •                                     | g = Floating-Point Precision, range is from zero to largest float value                                         |
| •                                     | Note: $a < b$ and $g < b - a$                                                                                   |
| Algorithm<br>Compute Length L (Bytes) |                                                                                                                 |
| 1.                                    | $Lbits = \text{ceiling}(\log_2(b-a)) - \text{floor}(\log_2(g)) + 1$ (note: +1 is for special values, see below) |
| 2.                                    | $L = \text{ceiling}(Lbits/8)$                                                                                   |
| 3.                                    | Follow steps in Starting Point B                                                                                |

### 7.1.2 Starting Point B: Specifying Length

| Input                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• a = Floating-Point Minimum, range is from smallest float value to largest float value</li> <li>• b = Floating-Point Maximum, range is from smallest float value to largest float value</li> <li>• L = Length of resulting mapped integer (note: this includes an extra bit for the special values, see below)</li> <li>• Note: <math>a &lt; b</math></li> </ul>               |
| Algorithm<br>Compute constants for forward and reverse mapping                                                                                                                                                                                                                                                                                                                                                         |
| <ol style="list-style-type: none"> <li>1. <math>bPow = \text{ceiling}(\log_2(b-a))</math></li> <li>2. <math>dPow = 8 * L - 1</math></li> <li>3. <math>sF = 2^{(dPow - bPow)}</math></li> <li>4. <math>sR = 2^{(bPow - dPow)}</math></li> <li>5. <math>Z_{offset} = 0.0</math></li> <li>6. if (<math>a &lt; 0</math> and <math>b &gt; 0</math>) then <math>Z_{offset} = sF * a - \text{floor}(sF * a)</math></li> </ol> |

## 7.2 Step 2: Forward and Reverse Mapping

Step 1 computes the scaling factors for the forward and reverse mapping in Step 2. These operations are computationally optimal, so the mapping requires only one multiply and two adds. Because the multiplication uses power of two values, an option is to implement the multiplication using floating point binary shifts (i.e., in hardware constrained environments).

### 7.2.1 Forward Mapping

| Input                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• <math>sF</math> = Forward scaling factor (computed in Step 1)</li> <li>• <math>a</math> = Floating-Point Minimum</li> <li>• <math>b</math> = Floating-Point Maximum</li> <li>• <math>Z_{\text{offset}}</math> = Zero-point offset (computed in Step 1)</li> <li>• <math>x</math> = floating-point value to map to integer <math>y</math></li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Algorithm<br>Compute mapped integer value $y$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <ol style="list-style-type: none"> <li>1. If <math>x</math> is a special value, then:             <ul style="list-style-type: none"> <li><math>y</math> = Table Lookup to map to specified bit pattern (see Section 7.2.3)</li> </ul> </li> <li>2. If <math>x &lt; a</math> then:             <ul style="list-style-type: none"> <li><math>y</math> = Special bit pattern IMAP_BELOW_MINIMUM (see Section 7.2.3.1)</li> </ul> </li> <li>3. If <math>x &gt; b</math> then:             <ul style="list-style-type: none"> <li><math>y</math> = Special bit pattern IMAP_ABOVE_MAXIMUM (see Section 7.2.3.1)</li> </ul> </li> <li>4. Else <math>x</math> is a normal floating-point number then:             <ol style="list-style-type: none"> <li>a. <math>y = \text{truncate}(sF * (x - a) + Z_{\text{offset}})</math> <ol style="list-style-type: none"> <li>i. Note: Since <math>sF</math> is a power of 2, an option is to use a floating-point shift instead of a multiplication</li> <li>ii. Note: Convert <math>y</math> into L number of bytes values (i.e., the lower L number of bytes is the “value” to send)</li> </ol> </li> </ol> </li> </ol> |
| Developer Notes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <ol style="list-style-type: none"> <li>1) IEEE 754 floating-point numbers do not explicitly represent the most significant 1-bit in the mantissa, so if performing a manual shift (i.e., the CPU does not support a floating-point shift), prepend a leading bit to the mantissa before shifting and converting to an integer. As a simple example, the IEEE 754 mantissa for 6 is 1000..., it first becomes 11000..., then ...00000110.</li> <li>2) If special values are not in use for a value, an optimization of the <u>forward mapping</u> is to skip the “special values check” and omit the table look-up. Preserve the extra bit (special value bit) in the data format but remove the conditional check (Step 1 above).</li> </ol>                                                                                                                                                                                                                                                                                                                                                                                                                |

## 7.2.2 Reverse Mapping

| Input                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• <math>sR</math> = Reverse scaling factor (computed in Step 1)</li> <li>• <math>a</math> = Floating-Point Minimum</li> <li>• <math>Z_{\text{offset}}</math> = Zero-point offset (computed in Step 1)</li> <li>• <math>y</math> = integer value to map to floating-point value <math>x</math></li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Algorithm<br>Compute mapped floating-point value $x$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <p>Define: <math>\text{Bit}(q,y)</math> = the <math>q^{\text{th}}</math> bit of <math>y</math>. <math>\text{msb}</math> = Most Significant Bit</p> <ol style="list-style-type: none"> <li>1. <math>\text{special\_value} = \text{Bit}(\text{msb},y) \ \&amp; \ \text{Bit}(\text{msb}-1,y)</math></li> <li>2. If <math>\text{special\_value}</math> equals 1 then <ol style="list-style-type: none"> <li>a. <math>x</math>=table lookup to map to specified floating-point value (Section see 7.2.3 Table 2 below)</li> </ol> </li> <li>3. Else <math>y</math> is a normal value then <ol style="list-style-type: none"> <li>a. <math>x = sR * (y - Z_{\text{offset}}) + a</math> <ol style="list-style-type: none"> <li>i. Note: Since <math>sR</math> is a power of 2, an option is to use a floating-point shift instead of a multiplication</li> </ol> </li> </ol> </li> </ol> |
| Developer Notes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <ol style="list-style-type: none"> <li>1) IEEE 754 floating-point numbers do not explicitly represent the most significant 1-bit in the mantissa, so if performing a manual shift (i.e., the CPU does not support a floating-point shift), prepend a leading bit to the mantissa before shifting and converting to an integer. As a simple example, the mantissa for 6 is 1000..., it first becomes 11000..., then ...00000110.</li> <li>2) Before applying this algorithm, verify the length is the correct length for the pre-computed <math>sR</math> value, see Section 7.4 for further information.</li> </ol>                                                                                                                                                                                                                                                               |

## 7.2.3 Special Value Mappings

In addition to normal numeric values this standard defines a list of special values to indicate unusual circumstances, such as a gimbal lock, division by zero, etc. This standard defines existing standardized values (based on IEEE-754) and a set of MISB definable values.

The first two bits indicate either a normal or special value as shown in Table 1. The  $\text{msb}$  is zero for all normal mapped values except for the maximum floating-point value,  $b$ , but only if  $(b-a)$ , as specified by the user, is a power of 2.

**Table 1: Bit Patterns**

| Bits           |           |                 | Description                                                                                                               |
|----------------|-----------|-----------------|---------------------------------------------------------------------------------------------------------------------------|
| $b_n$<br>(msb) | $b_{n-1}$ | $b_{n-2} - b_0$ |                                                                                                                           |
| 0              | x         | x               | Normal mapped value ( $x=1$ or $0$ )                                                                                      |
| 1              | 0         | 0               | Normal mapped value. This is the max value of the normal mapping values; this is the <u>only</u> normal value with msb=1. |
| 1              | 0         | x               | (At least one x bit = 1) – Reserved section of bit space                                                                  |
| 1              | 1         | x               | Special value indicator ( $x=1$ or $0$ ) – see Table 2                                                                    |

If both the first and second msb are set to one, then the third, fourth and fifth msbs indicate which special value it is, as shown in Table 2. Since the mapped integer has fewer bytes than the original floating-point number not all the source NaN Identifiers can be defined. The default NaN Identifier is a value with all zeros.

**Table 2: Special Value Bit Patterns**

| Name         | Most Significant Bits |                        |                     |                    |           | Other bits      | Description - IEEE-754 values<br>(or user defined)      |
|--------------|-----------------------|------------------------|---------------------|--------------------|-----------|-----------------|---------------------------------------------------------|
|              | $b_n$<br>(msb)        | $b_{n-1}$<br>(Special) | $b_{n-2}$<br>(Sign) | $b_{n-3}$<br>(NaN) | $b_{n-4}$ | $b_{n-5} - b_0$ |                                                         |
| +Infinity    | 1                     | 1                      | 0                   | 0                  | 1         | Zero Filled     | Positive Infinity ( $+\infty$ )                         |
| -Infinity    | 1                     | 1                      | 1                   | 0                  | 1         | Zero Filled     | Negative Infinity ( $-\infty$ )                         |
| +QNaN        | 1                     | 1                      | 0                   | 1                  | 0         | NaN Id*         | Positive Quiet NaN (Not a Number)                       |
| -QNaN        | 1                     | 1                      | 1                   | 1                  | 0         | NaN Id*         | Negative Quiet NaN                                      |
| +SNaN        | 1                     | 1                      | 0                   | 1                  | 1         | NaN Id*         | Positive Signal NaN – “Other bits” are the signal value |
| -SNaN        | 1                     | 1                      | 1                   | 1                  | 1         | NaN Id*         | Negative Signal NaN – “Other bits” are the signal value |
| MISB Defined | 1                     | 1                      | 1                   | 0                  | 0         | MISB Defined    | MISB Defined Special Values. See Section 7.2.3.1        |
| User Defined | 1                     | 1                      | 0                   | 0                  | 0         | User Defined    | “Other bits” enumerate user defined signals             |
| Reserved     | 1                     | 0                      | X                   | X                  | X         | X               | Reserved (At least one “X” is set to 1)                 |

\*This standard defines only one NaN Identifier, i.e., fill bits  $b_{n-5} - b_0$  with zeros. The MISB will define NaN Identifier's as necessary; please contact the MISB to define new values. The values need to be acceptable universally and not special values for a specific processor, application or program.

## ST 1201.5 Floating Point to Integer Mapping

This standard does not define a special value for the IEEE 754 negative zero value; use positive zero to represent negative zero values.

| Requirement  |                                                                                                       |
|--------------|-------------------------------------------------------------------------------------------------------|
| ST 1201.1-12 | All negative zero values shall be mapped to a positive zero and sent as a normal (non-special) value. |

In addition to the IEEE bit values a set of MISB-defined bit patterns are available for use within a Parent Document. For example, in MISB ST 0601 [2] the sensor depression angle value could include a bit pattern to indicate gimbal lock; this bit pattern would then only be valid for that value in ST 0601.

Developers note: Future versions of this standard may use one or more of the *Reserved* special values in Table 2. For forward-compatibility (i.e., reverse map a value which was forward mapped with newer standard implementation), implementations need to handle these “unknown” special cases (e.g., raise exception, set flags, etc.) preventing a software failure.

### 7.2.3.1 MISB Defined Special Values

The MISB defines special values for signaling information when IMAP encodes values, e.g., an IMAP overflow. Currently, the defined and RESERVED values need only one byte. Future expansions (i.e., RESERVED-GROWTH) will add bytes as needed. Table 3 lists the MISB Defined Special Values.

**Table 3: MISB Defined Special Values**

| Name               | Most Significant Bits<br>(From Table 2) |                               |                            |                           |                  | MISB Special<br>Value Bits |                  |                  | Other Bits  |
|--------------------|-----------------------------------------|-------------------------------|----------------------------|---------------------------|------------------|----------------------------|------------------|------------------|-------------|
|                    | b <sub>n</sub><br>(msb)                 | b <sub>n-1</sub><br>(Special) | b <sub>n-2</sub><br>(Sign) | b <sub>n-3</sub><br>(NaN) | b <sub>n-4</sub> | b <sub>n-5</sub>           | b <sub>n-6</sub> | b <sub>n-7</sub> |             |
| IMAP_BELOW_MINIMUM | 1                                       | 1                             | 1                          | 0                         | 0                | 0                          | 0                | 0                | Zero Filled |
| IMAP_ABOVE_MAXIMUM | 1                                       | 1                             | 1                          | 0                         | 0                | 0                          | 0                | 1                | Zero Filled |
| RESERVED           | 1                                       | 1                             | 1                          | 0                         | 0                | 0                          | 1                | 0                | Zero Filled |
| RESERVED           | 1                                       | 1                             | 1                          | 0                         | 0                | 0                          | 1                | 1                | Zero Filled |
| RESERVED           | 1                                       | 1                             | 1                          | 0                         | 0                | 1                          | 0                | 0                | Zero Filled |
| RESERVED           | 1                                       | 1                             | 1                          | 0                         | 0                | 1                          | 0                | 1                | Zero Filled |
| RESERVED           | 1                                       | 1                             | 1                          | 0                         | 0                | 1                          | 1                | 0                | Zero Filled |
| RESERVED - GROWTH  | 1                                       | 1                             | 1                          | 0                         | 0                | 1                          | 1                | 1                | RESERVED    |

The IMAP\_BELOW\_MINIMUM special value signals an IMAP encoding utilized a value less than the minimum value of the IMAP range (see Section 7.2.1 for how this applies in the encoding algorithm). For example, using IMAPB<sup>1</sup>(-50.0, 5000.0, 3) to encode the value -100.0 produces the IMAP\_BELOW\_MINIMUM value.

<sup>1</sup> See Section 7.3 for description of IMAPB notation.

The IMAP\_ABOVE\_MAXIMUM special value signals an IMAP encoding utilizing a value greater than the maximum value of the IMAP range (see Section 7.2.1 for how this applies in the encoding algorithm). For example, using IMAPB(-50.0, 5000.0,3) to encode the value 6000.0 produces the IMAP\_ABOVE\_MAXIMUM value.

Invoking standards define if and how to use the IMAP\_BELOW\_MINIMUM or IMAP\_ABOVE\_MAXIMUM on a case-by-case basis (e.g., Local Set item by item). When this information is not provided, the invoking standard uses the default behavior as follows:

- The default behavior after reverse mapping a value with the result IMAP\_BELOW\_MINIMUM is to set the resulting value to the minimum range of the IMAP encoding.
- The default behavior after reverse mapping a value with the result IMAP\_ABOVE\_MAXIMUM is to set the value to the maximum range of the IMAP encoding.

The RESERVED rows in Table 3 are for future MISB special values. The RESERVED-GROWTH row in Table 3, with bits  $b_{n-5}$ ,  $b_{n-6}$ , and  $b_{n-7}$  all equal to one (1), provides a set of future special values using the “Other Bits”.

Developers note: Future versions of this standard may use one or more of the *Reserved* special values in Table 3. For forward-compatibility (i.e., reverse map a value which was forward mapped with a newer standard implementation), implementations need to handle these “unknown” special cases (e.g., raise exception, set flags, etc.) preventing a software failure.

### 7.3 MISB and MISB Document Notation

When using or “invoking” this standard in a MISB Parent Document it is important to be clear as to the starting point, and the values used as “inputs.” The notation to use in a Parent Document to signal the use of forward mapping a value is either IMAPA or IMAPB. The notation to use when reverse mapping of a previously encoded IMAP value is RIMAPB. RIMAPA is unnecessary since the byte array to reverse map has a defined length (i.e., not computed from a precision).

| Requirement(s) |                                                                                                                                                                     |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ST 1201.1-13   | When the float precision is fixed within the implementing standard, the notation for Starting Point A shall be IMAPA (<min float>, <max float>, <float precision>). |
| ST 1201.2-16   | When the float precision is computed or defined at runtime, the notation for Starting Point A shall be IMAPA (<min float>, <max float>).                            |
| ST 1201.1-14   | When the length is fixed within the implementing standard, the notation for Starting Point B shall be IMAPB (<min float>, <max float>, <length bytes>).             |
| ST 1201.2-17   | When the length is computed or defined at runtime, the notation for Starting Point B shall be IMAPB (<min float>, <max float>).                                     |
| ST 1201.1-15   | When adding user defined bit patterns, they shall be listed immediately after the IMAPA or IMAPB notation.                                                          |
| ST 1201.5-18   | When reverse mapping an IMAP value, the notation shall be RIMAPB (<min float>, <max float>).                                                                        |

Example: Starting Point A:

- IMAPA(-200.0, 3000.0, 0.5)
- Interpretation: map a value in the range from -200.0 to 3000.0 using, at the minimum, increments of 0.5.

Example: Starting Point A (with computed precision):

- IMAPA(-200.0, 3000.0)
- Interpretation: map a value in the range from -200.0 to 3000.0 using a precision computed at runtime.

Example: Starting Point B:

- IMAPB(-200.0, 3000.0, 3)
- Interpretation: map a value in the range from -200.0 to 3000.0 using 3 bytes (see Section 7.4 for comments on length).

Example: Starting Point B (with computed length):

- IMAPB(-200.0, 3000.0)
- Interpretation: map a value in the range from -200.0 to 3000.0 using a length computed at runtime (see Section 7.4 for comments on length).

Example: Reverse Mapping :

- RIMAPB(-200.0, 3000.0)
- Interpretation: reverse map a byte array value in the range from -200.0 to 3000.0 using a length determined from the byte array size.

## 7.4 Length Processing

Lengths computed or provided when defining the mapping (IMAPA, IMAPB) are the recommended number of bytes to use. Depending on the form used to transmit KLV data (Universal Sets, Local Sets, Individual Items), it is possible to use a different [KLV] length (L) instead of the recommended or computed [IMAP] length. When using a different length, it is important to compute the constants needed to do the forward and reverse mapping based on the KLV supplied length.

## 8 Algorithm Development – Informative

The development of the algorithm for this standard is discussed in the following sections:

- 1) Goal
- 2) Mapping and Inverse Mapping
- 3) Error Analysis
- 4) Simplification
- 5) Zero Matching
- 6) Computing d
- 7) Power of 2 adjustments for b
- 8) Computing L from Precision

## 9) Summary

**8.1 Goal**

The goal is to map a floating-point range ( $F_R$ ) to an integer range ( $I_R$ ) with a chosen maximum precision ( $g$ ) using the smallest integer range ( $I_R$ ) and the least amount of computation. In addition, when a  $F_R$  range includes zero the algorithm must map zero to a specific integer value (zero offset), and all negative floating-point values must map to values less than the zero offset. Refer to Section 6 for the requirements for this algorithm.

Examples:

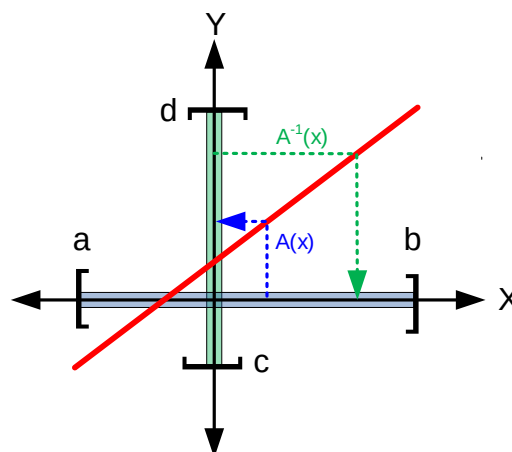
- 1) Map floating-point range 0.1 to 3.1 to 2 bytes
- 2) Map -1.0 to +1.0 to 2 bytes
- 3) Map -0.5 to -0.3 to 1 byte
- 4) Map -3.14159265 to 3.14159265 to 4 bytes
- 5) Map floating-point range 0.1 to 3.1 with precision of 0.01 or better
- 6) Map -1.0 to +1.0 to 2 bytes with precision of 0.001 or better
- 7) Map -0.5 to -0.3 to 1 byte with precision of 0.01 or better
- 8) Map -3.14159265 to 3.14159265 with precision of 0.00314159265 or better

**8.2 Mapping and Inverse Mapping Definition**

Given a  $F_R = \{x \in \mathbb{R} | x \in [a, b]\}$  and  $I_R = \{y \in \mathbb{Z} | y \in [c, d]\}$

Let  $A(x)$  be a linear mapping from  $F_R$  to  $I_R$  and  $A^{-1}(y)$  be a mapping from  $I_R$  to  $F_R$ .

Figure 2 illustrates the linear mapping, where  $A(x)$  (blue dotted line) maps from  $F_R$  (x-axis) to  $I_R$  (y-axis) via the red line;  $A^{-1}(y)$  (green dotted line) maps from the  $I_R$  (y-axis) to  $F_R$  (x-axis) via the red line.



**Figure 2:  $A(x)$  maps  $F_R$  to  $I_R$ ;  $A^{-1}(x)$  maps  $I_R$  to  $F_R$**

Using the two-point method for defining the mapping:

$$(Eq\ 1) \quad y - y_1 = \frac{y_2 - y_1}{x_2 - x_1} (x - x_1) = y - c = \frac{d - c}{b - a} (x - a)$$

After rearranging, since  $y$  is an integer, truncate or round the right-hand side of the equation before computing  $y$ :

$$(Eq\ 2) \quad y = I\left(\frac{d - c}{b - a} (x - a) + c\right)$$

Function  $I(\ )$  is either the truncation or rounding function (discussed in Section 8.3). (Eq 2) equation defines  $A(x)$ , the forward mapping of any real value (floating point) range to an integer range.

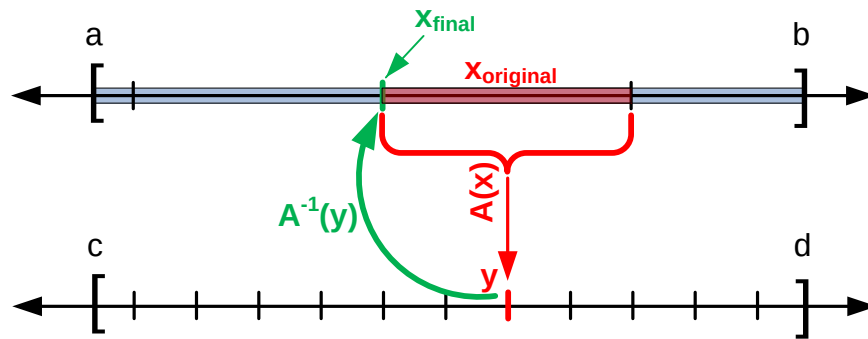
(Eq 3) defines the inverse of (Eq 2)  $A^{-1}(y)$ :

$$(Eq\ 3) \quad x = (y - c) \frac{b - a}{d - c} + a$$

The above equations are for all  $a, b, c$ , and  $d$  values; however, the following sections describe additional simplification and improvements.

### 8.3 Error Analysis

The mapping function is not a one-to-one function, which means the mapping and subsequent inverse mapping introduce errors. For each value  $y$ , in  $I_R$ , a unique range of  $x$  values in  $F_R$  maps to a single  $y$  value; define this range of  $x$  values as  $x_{original}$ . When the integer value  $y$  is inverse mapped back to an  $x$  value it will only match one of the values of  $x_{original}$ . Therefore, the forward mapping of an  $x$  value to an integer, followed by the reverse mapping back to a floating-point  $x$  value induces an error (except in special cases). The tolerance of this error is dependent on the application, so the implementer of this standard must ensure the use of large enough  $c$  and  $d$  values to create acceptable minimum error – more on this topic in other sections below. Figure 3 shows a subset of values,  $x_{original}$  (in red), mapping to a single integer value  $y$ . This also shows the single integer value,  $y$ , maps back to only one value,  $x_{final}$  (green), in the floating-point range



**Figure 3: Many-to-one forward mapping and one-to-one reverse mapping**

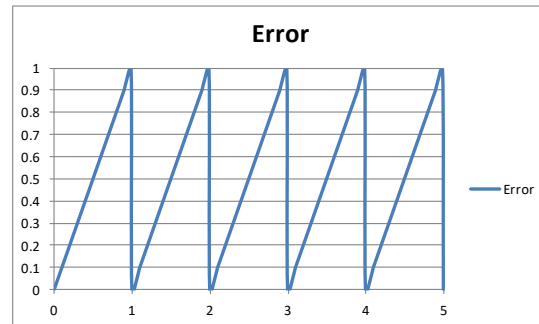
#### Example 1:

$$\text{Let } a=0.0, b=5.0 \text{ and } c=0, d=5; \text{ then } y = I\left(\frac{5-0}{5.0-0.0} (x - 0.0) + 0\right) = I\left(\frac{5}{5} x\right) = I(x)$$

For this example, use  $I(x) = \text{truncate}(x)$ ; therefore, all values of the  $x_{\text{original}}$  range map to a single  $y$  value. The inverse mapping, maps  $y$  to a single  $x$  value,  $x_{\text{final}}$ . For example, all values in  $x_{\text{original}} = \{x \in [1.0, 2.0)\}$  map to  $y=1$ , and the inverse mapping value of  $y=1$  map to  $x_{\text{final}} = 1.0$  for all values in the  $x_{\text{original}}$  range.

The error is the absolute difference between the starting value and the result of forward and inverse mapping. For example, an  $x$  value of 1.4 maps to  $y=1$ , then inverse maps back to  $x=1.0$ , producing an error of  $|1.4 - 1.0| = 0.4$ .

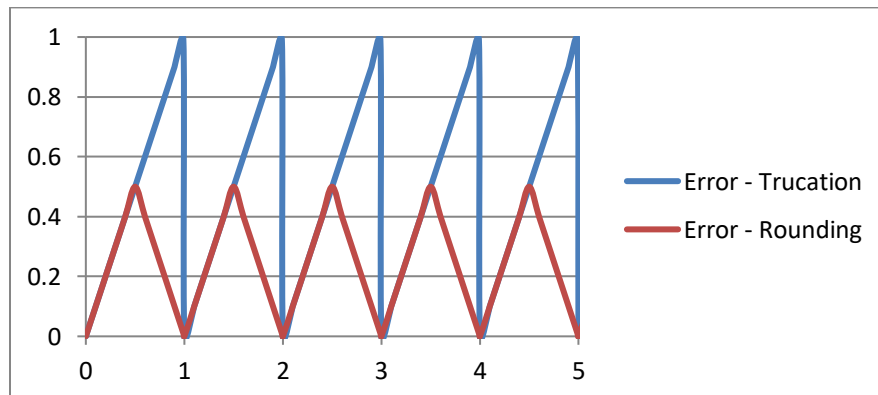
To compute the amount of error:  $E(x) = |x - A^{-1}(A(x))|$ . Graphing the error for Example 1 produces the saw-tooth graph shown in Figure 4.



**Figure 4: Error in forward mapping of Example 1**

The maximum error in Example 1 is slightly less than 1.0; thus, implementors may use this mapping function if this level of error is acceptable. If less error is desired, it is a simple matter to use a larger integer range; for example, doubling the range cuts the error in half.

There are multiple ways of converting a floating-point number to an integer, including “normal” rounding, truncating, round away from zero, round to zero, etc. Normal rounding rounds down when the fractional value is less than 0.5 and rounds up when the fractional value is 0.5 or greater. Considering error, the best conversion technique is normal rounding shown in Figure 5.



**Figure 5: Rounding versus Truncation error**

However, to support the zero matching requirements a truncation must be used instead (see Section 8.5), so (Eq 2) is changed to include truncation. Since  $x$  ranges from  $[a, b]$ , all values of

$(x-a)$  will be greater than or equal to zero so **truncate** and **floor** perform the same function. (Eq 2) uses the **floor** notation  $\lfloor \cdot \rfloor$  throughout the remainder of this section:

$$(Eq\ 4) \quad y = \left\lfloor \frac{d-c}{b-a} (x - a) + c \right\rfloor$$

## 8.4 Simplifications

There are two simplifications to (Eq 4) to reduce overall (floating point) errors and enable some optimizations: shifting the integer range and shifting the floating-point range.

The integer range ( $I_R$ ) values,  $c$  and  $d$ , are arbitrary for this algorithm, so in this standard the minimum integer value is forced to always equal zero. When  $c$  is zero,  $d$  is the maximum value desired by the implementer for the desired precision. An additional benefit with the minimum value equal to zero is the resulting value is never negative, therefore it is an unsigned integer requiring no sign bit (nor 2's complement formatting). Setting  $c=0$  results in the following change to (Eq 4):

$$(Eq\ 5) \quad y = \left\lfloor \frac{d}{b-a} (x - a) \right\rfloor$$

Using substitution, letting  $x'=x-a$ , shifts the floating-point range,  $F_R$ . Shifting the range  $F_R$  by a constant ( $a$ ) produces a new range called  $F'_R$ .  $F'_R = \{x' \in \mathbb{R} | x' \in [0, b']\}$  where  $a'=a-a=0$  and  $b' = b-a$ . With this change the minimum value of  $F'_R$  is zero, so (Eq 5) and (Eq 3) are respectively simplified to:

$$(Eq\ 6) \quad y = \left\lfloor \frac{d}{b'} x' \right\rfloor = \left\lfloor \frac{d}{b'} (x - a) \right\rfloor$$

$$(Eq\ 7) \quad x = \frac{b'}{d} y + a$$

## 8.5 Zero Matching

When the floating-point range  $F'_R$  includes zero it is desirable to have the zero value directly map to an integer in  $I_R$ . This zero-integer value is the “Zero Point” or  $I_{zero}$ . Furthermore, all negative values in  $F_R$  map to integers less than  $I_{zero}$  and all positive values map to integers equal to or greater than  $I_{zero}$ .

### Example 2:

With  $F_R = -0.9$  to  $1.1$  and  $d=11$ , Table 4 lists example  $x$  values in column 1. Column 2 is the  $x$  value adjusted to the new  $F'_R$  range,  $x'$ . Column 3 is the mapping before converting to an integer. Columns 4 through 7 show different integer conversions of the mapping. Note:  $g=2.0/11 = 0.1818$ .

**Table 4: Various mappings for Example 2**

| 1        | 2        | 3        | 4       | 5     | 6        | 7     |
|----------|----------|----------|---------|-------|----------|-------|
| X        | x'       | (d/b')x' | ceiling | floor | truncate | round |
| -0.9     | 0        | 0        | 0       | 0     | 0        | 0     |
| -0.71818 | 0.181818 | 1        | 1       | 1     | 1        | 1     |
| -0.53636 | 0.363636 | 2        | 2       | 2     | 2        | 2     |
| -0.35455 | 0.545455 | 3        | 3       | 3     | 3        | 3     |
| -0.17273 | 0.727273 | 4        | 4       | 4     | 4        | 4     |
| -0.08182 | 0.818182 | 4.5      | 5       | 4     | 4        | 5     |
| 0        | 0.9      | 4.95     | 5       | 4     | 4        | 5     |
| 0.009091 | 0.909091 | 5        | 5       | 5     | 5        | 5     |
| 0.1      | 1        | 5.5      | 6       | 5     | 5        | 6     |
| 0.190909 | 1.090909 | 6        | 6       | 6     | 6        | 6     |
| 0.372727 | 1.272727 | 7        | 7       | 7     | 7        | 7     |
| 0.554545 | 1.454545 | 8        | 8       | 8     | 8        | 8     |
| 0.736364 | 1.636364 | 9        | 9       | 9     | 9        | 9     |
| 0.918182 | 1.818182 | 10       | 10      | 10    | 10       | 10    |
| 1.1      | 2        | 11       | 11      | 11    | 11       | 11    |

To illustrate what happens when not using a Zero Point, the red row in Table 4 shows the original zero (0) value (in column 1) and the value equal to 0.9 (in column 2) after shifting the floating-point range. The 0.9 value maps to 4.95 (column 3) before conversion to an integer. Based on the integer conversion technique (columns 4-7), the original zero value maps to a value of 4 or 5, which is the same value as a preceding negative value (in yellow) of -0.08182. When mapping the integer value back to a floating-point value the original value of zero (in column 1) becomes -0.08182; a zero value becomes negative after the mapping. This behavior is undesirable; instead, the zero in  $F_R$  should map to an integer that maps back exactly to zero in  $F_R$ .

To achieve this, the zero in  $F_R$  must map exactly to a whole integer (i.e., before converting to an integer) – this integer value is the Zero Point,  $I_{\text{zero}}$ . With the Zero Point, all negative values in  $F_R$  map to integers less than  $I_{\text{zero}}$ . This means all negative values forward and reverse map back to negative values; the zero value forward and reverse maps back to zero; and all positive values forward and reverse map back to positive values or zero (if they are close to zero).

To implement the Zero Point the whole mapping equation is shifted, so the zero in  $F_R$  becomes an integer value ( $I_{\text{zero}}$ ); an offset is added before the integer conversion. Furthermore, truncation (or floor) ensures all negative values are below  $I_{\text{zero}}$ ; conversely, ceiling and rounding will force negative values to be identical to  $I_{\text{zero}}$ . The Zero Point shift is different than changing the bounds (a,b or c,d) of the equations; the slope of the linear mapping is not affected by this change.

The adjustment to the mapping equations (Eq 5) and (Eq 6) are:

$$\text{(Eq 8)} \quad Z_{\text{offset}} = \frac{d}{b'}a - \left\lfloor \frac{d}{b'}a \right\rfloor, \text{ Note: } 0.0 \leq Z_{\text{offset}} < 1.0$$

$$(Eq\ 9) \quad y = \left\lfloor \frac{d}{b'} (x - a) + Z_{\text{offset}} \right\rfloor$$

$$(Eq\ 10) \quad x = \frac{b'}{d} (y - Z_{\text{offset}}) + a$$

Re-computing Example 2 with these changes results in:

$$Z_{\text{offset}} = \frac{11}{2.0}(-0.9) - \left\lfloor \frac{11}{2.0}(-0.9) \right\rfloor = -4.95 - (-5.0) = 0.05$$

Table 5 shows the mapping with the  $Z_{\text{offset}}$  (0.05) that correlates to the Zero Point ( $I_{\text{zero}}$ ), value of 5. All negative  $x$  values map to 4 or less and all positive values (and zero) map to 5 or greater. Do not use ceiling and rounding; as shown in column 4 and 7 they map positive and zero values with negative value.

**Table 5: Example 2 recomputed using  $Z_{\text{offset}}$**

| 1        | 2        | 3                              | 4       | 5     | 6        | 7     |
|----------|----------|--------------------------------|---------|-------|----------|-------|
| $x$      | $x'$     | $(d/b')x' + Z_{\text{offset}}$ | ceiling | floor | truncate | round |
| -0.9     | 0        | 0.05                           | 1       | 0     | 0        | 0     |
| -0.71818 | 0.181818 | 1.05                           | 2       | 1     | 1        | 1     |
| -0.53636 | 0.363636 | 2.05                           | 3       | 2     | 2        | 2     |
| -0.35455 | 0.545455 | 3.05                           | 4       | 3     | 3        | 3     |
| -0.17273 | 0.727273 | 4.05                           | 5       | 4     | 4        | 4     |
| -0.08182 | 0.818182 | 4.55                           | 5       | 4     | 4        | 5     |
| 0        | 0.9      | 5                              | 5       | 5     | 5        | 5     |
| 0.009091 | 0.909091 | 5.05                           | 6       | 5     | 5        | 5     |
| 0.1      | 1        | 5.55                           | 6       | 5     | 5        | 6     |
| 0.190909 | 1.090909 | 6.05                           | 7       | 6     | 6        | 6     |
| 0.372727 | 1.272727 | 7.05                           | 8       | 7     | 7        | 7     |
| 0.554545 | 1.454545 | 8.05                           | 9       | 8     | 8        | 8     |
| 0.736364 | 1.636364 | 9.05                           | 10      | 9     | 9        | 9     |
| 0.918182 | 1.818182 | 10.05                          | 11      | 10    | 10       | 10    |
| 1.1      | 2        | 11.05                          | 12      | 11    | 11       | 11    |

## 8.6 Computing $d$

In all the previous equations, the number of bytes the user specifies with value  $L$  determines the value  $d$ . The value  $d$  uses all the bits in the bytes including the most significant bit (msb); however, there is only one case which uses the msb for a mapped value. When the user specifies a maximum value,  $b$ , which is a power of 2, the mapped integer value can include the msb. In this case the only time the msb is used is when the floating-point value to map is the maximum value. When this happens, the msb is set to 1 and all other bits are set to zero. All other uses of the msb signal special values, see Section 7.2.3.

$$(Eq\ 11) \quad N_{\text{bits}} = 8 * L - 1$$

$$(Eq\ 12) \quad d = 2^{N_{bits}}$$

(Eq 12) shows  $d$  is a power of two, which affords an optimization capability described in Section 8.7.

### 8.7 Power of 2 Adjustment for $b$

To potentially improve algorithm efficiency and reduce floating-point rounding errors, this standard adjusts the floating-point range,  $F_R$ , to be a power of two. The cost of this adjustment is a slight reduction in the precision.

Examining the equation for the mapping shows the mapping is primarily a simple division, multiplication, and addition:  $y = \left\lfloor \frac{d}{b'}(x - a) + Z_{offset} \right\rfloor$ . Floating-point division and multiplication can easily introduce small numeric floating-point rounding errors. To reduce this error and to provide a more efficient algorithm  $b'$  is adjusted to be based on a power of two.

$$(Eq\ 13) \quad \bar{b} = 2^{\lceil \log_2 b' \rceil}$$

With the adjustments to  $b'$  (Eq 9) and (Eq 10) are updated as follows:

$$(Eq\ 14) \quad y = \left\lfloor \frac{d}{\bar{b}}(x - a) + Z_{offset} \right\rfloor$$

$$(Eq\ 15) \quad x = \frac{\bar{b}}{d}(y - Z_{offset}) + a$$

Since  $d$  and  $\bar{b}$  are both powers of two, an optimization is to pre-compute the forward and reverse scaling factors:

$$(Eq\ 16) \quad \frac{d}{\bar{b}} = S_F = 2^{(N_{bits} - \lceil \log_2 b' \rceil)}$$

$$(Eq\ 17) \quad \frac{\bar{b}}{d} = S_R = 2^{(\lceil \log_2 b' \rceil - N_{bits})}$$

Adjusting (Eq 14) and (Eq 15) produces the final mapping equations:

$$(Eq\ 18) \quad y = \lfloor S_F(x - a) + Z_{offset} \rfloor$$

$$(Eq\ 19) \quad x = S_R(y - Z_{offset}) + a$$

### 8.8 Computing $L$ from known precision

When using this standard there are two starting points based either on: (Starting Point A) the minimum, maximum and known floating-point precision, or (Starting Point B) the minimum, maximum and desired length. The preceding sections describe the steps for computing scaling factors based on Starting Point B, knowing the minimum, maximum and number of bytes ( $L$ ). This section shows the computation of  $L$  based on the known minimum, maximum and known floating-point precision (Starting Point A).

Because the scaling factors ( $S_F$  and  $S_R$ ) in (Eq 18) and (Eq 19) are rounded up to powers of two, the users precision value,  $g$ , is rounded down (providing more precision) to a power of two,  $g'$ , to ensure the users required precision is maintained in the mapping.

$$(Eq\ 20) \quad g' = 2^{\lfloor \log_2 g \rfloor}$$

Given  $a$ ,  $b$ , and  $g'$ ; where  $a$  is the minimum value of the range,  $b$  is the maximum value of the range, and  $g'$  is the floating-point precision rounded down, the number of bits,  $L_{bits}$ , needed to represent this number for the mapping is:

$$(Eq\ 21) \quad L_{bits} = \left\lceil \log_2 \left( \frac{b-a}{g'} \right) \right\rceil + 1 \quad \text{Note: } +1 \text{ is for special values bit.}$$

Combining equations (Eq 20) and (Eq 21) provides a single computation for the bits in (Eq 22). (Eq 23) shows the computation of the number of bytes from  $L_{bits}$ .

$$(Eq\ 22) \quad L_{bits} = \lceil \log_2(b - a) \rceil - \lfloor \log_2 g \rfloor + 1$$

$$(Eq\ 23) \quad L = \left\lceil \frac{L_{bits}}{8} \right\rceil$$

(Eq 11) uses the value of  $L$ .

## 8.9 Summary

The following shows a consolidation of all the equations needed to map values to and from floating point to integer.

### Starting Point A: User specifies $a$ , $b$ , and $g$ (the desired precision to use)

One-time computation:

$$L_{bits} = \lceil \log_2(b - a) \rceil - \lfloor \log_2 g \rfloor + 1$$

$$L = \left\lceil \frac{L_{bits}}{8} \right\rceil$$

Now  $a$ ,  $b$  and  $L$  are defined for starting point B.

### Starting Point B: User specifies, $a$ , $b$ , and $L$ (number of bytes to use)

One-time computation:

$$b_{pow} = \lceil \log_2(b - a) \rceil$$

$$d_{pow} = 8 * L - 1 \quad (\text{Number of bits for } d)$$

$$S_F = 2^{(d_{pow} - b_{pow})} \quad (\text{Scaling for forward mapping, note same as bitwise shift})$$

$$S_R = 2^{(b_{pow} - d_{pow})} \quad (\text{Scaling for reverse mapping, note same as bitwise shift})$$

$$Z_{offset} = 0 \quad (\text{Default offset to zero})$$

$$\text{If } (a < 0 \text{ and } b > 0) \text{ then } Z_{offset} = S_F a - \lfloor S_F a \rfloor \quad (\text{only compute if condition met})$$

## ST 1201.5 Floating Point to Integer Mapping

Once computing the above values, use  $S_F$ ,  $S_R$ , and  $Z_{\text{offset}}$  for forward mapping and reverse mapping every  $x$  and  $y$  value, respectively.

Per  $x$  value:

$$y = \lfloor S_F(x - a) + Z_{\text{offset}} \rfloor$$

Per  $y$  value:

$$x = S_R(y - Z_{\text{offset}}) + a$$

See Section 7.2.3 for information on handling special values (i.e., infinity, NaN, etc.) of  $x$  and  $y$ .

## 9 Deprecated Requirements

The following requirements are deprecated because they are not testable.

| Requirement(s)               |                                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ST 1201.1-01<br>(Deprecated) | The binary representation of a floating-point value shall be in the format of an IEEE 754-2008 floating-point value.                                                                                                                                                                                                                                                            |
| ST 1201.1-02<br>(Deprecated) | The algorithm shall perform the mapping based on a [user] specified floating-point range from a min value to a max value – inclusive (including the end points).                                                                                                                                                                                                                |
| ST 1201.1-03<br>(Deprecated) | The algorithm shall be a linear mapping from a floating-point value to a non-negative integer value of [user] defined length in bytes.                                                                                                                                                                                                                                          |
| ST 1201.1-04<br>(Deprecated) | The binary representation of a non-negative integer value shall be in the format of a standard unsigned integer, with the length, in bytes, defined by the [user's] implementing Parent Document.                                                                                                                                                                               |
| ST 1201.1-05<br>(Deprecated) | The algorithm shall provide an inverse mapping from a non-negative integer value to the original floating-point value within a [user] specified precision. (Note: the mapping algorithm can supply better precision than what the user specifies. The defined length and precision are related – given one the other can be computed but both cannot be specified by the user). |
| ST 1201.1-06<br>(Deprecated) | If the [users] floating-point range includes 0.0, then the algorithm shall map 0.0 exactly to a specific integer value, called a “zero offset.”                                                                                                                                                                                                                                 |
| ST 1201.1-07<br>(Deprecated) | All floating-point values that are negative shall map to an integer that is less than the zero offset.                                                                                                                                                                                                                                                                          |
| ST 1201.1-08<br>(Deprecated) | All values that are positive shall map to values greater than or equal to the zero offset.                                                                                                                                                                                                                                                                                      |
| ST 1201.1-09<br>(Deprecated) | The algorithm shall be designed to be as few operations as possible (during the actual mapping) for computational efficiency.                                                                                                                                                                                                                                                   |
| ST 1201.1-10<br>(Deprecated) | The resulting format shall provide a set of special values which map to the IEEE 754-2008 special values, including: $\pm$ infinity, $\pm$ QNaN and $\pm$ SNan (note: negative zero is not included in this requirement).                                                                                                                                                       |
| ST 1201.1-11<br>(Deprecated) | The resulting format shall provide a bit space for custom signals to be sent by the user as defined by the [users] implementing Parent Document (e.g., MISB ST 0601).                                                                                                                                                                                                           |

## 10 Appendix A – Usage Examples and Test Values

Table 6 shows examples of ranges, precisions, and number of bytes along with the invoking document IMAP notation.

**Table 6: Notation examples**

| Name        | Min         | Max        | Precision     | Number of Bytes | IMAP Statement                                |
|-------------|-------------|------------|---------------|-----------------|-----------------------------------------------|
| Altitude    | -900.0      | 19000.0    | Don't care    | 2               | IMAPB(-900, 19000, 2)                         |
| Covariance  | -1.0        | 1.0        | Don't care    | 2               | IMAPB(-1, 1, 2)                               |
| Radians     | -3.14159265 | 3.14159265 | Don't care    | 3               | IMAPB(-3.14159265, 3.14159265, 3)             |
| Altitude    | -900.0      | 19000.0    | 0.5           | Don't care      | IMAPA(-900, 19000, 0.5)                       |
| Covariance  | -1.0        | 1.0        | .0001         | Don't care      | IMAPA(-1, 1, .0001)                           |
| Radians     | -3.14159265 | 3.14159265 | 0.00314159265 | Don't care      | IMAPA(-3.14159265, 3.14159265, 0.00314159265) |
| Small Range | 0.1         | 0.9        | Don't care    | 2               | IMAPB(0.1, 0.9, 2)                            |

### 10.1 Detailed Usage Example

The following illustrates a complete example of the computations for mapping an altitude range using, IMAPA(-900, 19000, 0.5).

Example 3: Altitude (Starting point A):

- a = Floating-Point Minimum = -900.0
- b = Floating-Point Maximum = 19,000.0
- g = Floating-Point Precision = 0.5

Starting Point A - Algorithm: Compute Length L (Bytes)

1. Lbits = **ceiling**( $\log_2(b-a)$ )-**floor**( $\log_2(g)$ )+1  
= **ceiling**( $\log_2(19,000-(-900))$ )-**floor**( $\log_2(0.5)$ )+1 = 17
2. L = **ceiling**(Lbits/8) = **ceiling**(17/8) = 3
3. Follow steps in Starting Point B

Starting Point B - Algorithm: Compute sF, sR and Z<sub>offset</sub>

1. bPow = **ceiling**( $\log_2(b-a)$ ) = **ceiling**( $\log_2(19,000-(-900))$ ) = 15
2. dPow = 8\*L-1 = 8\*3-1 = 23
3. sF = 2<sup>(dPow - bPow)</sup> = 2<sup>(23-15)</sup> = 2<sup>8</sup>
4. sR = 2<sup>(bPow - dPow)</sup> = 2<sup>(15-23)</sup> = 2<sup>(-8)</sup>
5. Z<sub>offset</sub> = 0.0
6. if (a<0 and b>0) then Z<sub>offset</sub> = sF\*a-**floor**(sF\*a)

## ST 1201.5 Floating Point to Integer Mapping

$$= (2^8)*(-900) - \text{floor}((2^8)*(-900)) = 0.0$$

*Forward Mapping:* Let  $x = 10.0$  (underlined section is the algorithm exercised)

1. If  $x$  is a special value, then:

$y = \text{Table Lookup to map to specified bit pattern}$

2. Else  $x$  is a normal floating-point number then:

$$y = \text{truncate}(sF*(x-a) + Z_{\text{offset}}) = \text{truncate}(2^8*(10.0 - (-900)) + 0.0) = 232,960 \\ = 0x03\ 8E00$$

*Reverse Mapping:* Let  $y = 232,960 = 0x03\ 8E00$

Define:  $\text{Bit}(q, y) = \text{the } q^{\text{th}} \text{ bit of } y$ . msb = Most significant bit

1.  $\text{special\_value} = \text{Bit}(\text{msb}, y) \ \& \ \text{Bit}(\text{msb}-1, y) = 0 \ \& \ 0$
2. If  $\text{special\_value}$  equals 1 then:  
 $x = \text{Table Lookup to map to specified floating-point value}$

3. Else  $y$  is a normal value then:

$$x = sR*(y - Z_{\text{offset}}) + a = (2^{(-8)}) * (232,960 - 0.0) + (-900) = 10.0$$

Table 7 shows some forward mapping and reverse mapping values for this example.

**Table 7: Forward/Reverse mapping for Example 3**

| x         | y       | y (hex)   | x mapped back |
|-----------|---------|-----------|---------------|
| -900.0    | 0       | 0x00 0000 | -900.0        |
| 0.0       | 230,400 | 0x03 8400 | 0.0           |
| 10.0      | 232,960 | 0x03 8E00 | 10.0          |
| -infinity | See hex | 0xE8 0000 | -infinity     |

The following illustrates a complete example of the computations for mapping the small range using, IMAPB(0.1, 0.9, 2).

Example 4: Small range (Starting point B):

- $a = \text{Floating-Point Minimum} = 0.1$
- $b = \text{Floating-Point Maximum} = 0.9$
- $L = 2 \text{ bytes}$

Starting Point B - Algorithm: Compute  $sF$ ,  $sR$  and  $Z_{\text{offset}}$

1.  $bPow = \text{ceiling}(\log_2(b-a)) = \text{ceiling}(\log_2(0.9-(0.1))) = 0$
2.  $dPow = 8*L-1 = 8*2-1 = 15$
3.  $sF = 2^{(dPow - bPow)} = 2^{(15-0)} = 2^{15}$
4.  $sR = 2^{(bPow - dPow)} = 2^{(0-15)} = 2^{(-15)}$
5.  $Z_{\text{offset}} = 0.0$
6. if  $(a < 0 \text{ and } b > 0)$  then  $Z_{\text{offset}} = sF*a - \text{floor}(sF*a)$

*Forward Mapping:* Let  $x = 0.5$  (underlined section is the algorithm exercised)

1. If  $x$  is a special value, then:

$y = \text{Table Lookup to map to specified bit pattern}$

2. Else  $x$  is a normal floating-point number then:

$$y = \text{truncate}(sF*(x-a)+Z_{\text{offset}}) = \text{truncate}(2^{15}*(0.5-(0.1))+0.0) = 13,107 \\ = 0x3333$$

*Reverse Mapping:* Let  $y = 13,107 = 0x3333$

Define:  $\text{Bit}(q,y)$  = the  $q^{\text{th}}$  bit of  $y$ . msb = Most significant bit

1.  $\text{special\_value} = \text{Bit}(\text{msb},y) \ \& \ \text{Bit}(\text{msb}-1,y) = 0 \ \& \ 0$

2. If  $\text{special\_value}$  equals 1 then:

$x = \text{Table Lookup to map to specified floating-point value}$

3. Else  $y$  is a normal value then:

$$x = sR*(y-Z_{\text{offset}})+a = (2^{(-15)}) * (13,107-0.0) + (0.1) = 0.499993896484375$$

Table 8 shows some forward mapping and reverse mapping values for this example.

**Table 8: Forward/Reverse mapping for Example 4**

| x         | y       | y (hex) | x mapped back     | Error   |
|-----------|---------|---------|-------------------|---------|
| 0.1       | 0       | 0x0000  | 0.1               | 0.0     |
| 0.5       | 13,107  | 0x3333  | 0.499993896484375 | 6.10e-6 |
| 0.9       | 26,214  | 0x6666  | 0.89998779296875  | 1.22e-5 |
| -infinity | See hex | 0xE800  | -infinity         | n/a     |

## 10.2 Test Values

Example test values for software development validation.

### 10.2.1 Test 1

| IMAP <sub>A</sub> (0.0, 100.0, 1E-5) |                                         |
|--------------------------------------|-----------------------------------------|
| IMAP Details:                        |                                         |
| Min                                  | = 0.0                                   |
| Max                                  | = 100.0                                 |
| Specified Precision                  | = 1.0E-5                                |
| Adjusted Precision                   | = 7.62939453125E-6                      |
| SF                                   | = 1.6777216E7                           |
| SR                                   | = 5.9604644775390625E-8                 |
| Length                               | = 4                                     |
| Zoffset                              | = 0.0                                   |
| Actual Precision                     | = 0.00000005960464477539 (5.960464e-08) |

## ST 1201.5 Floating Point to Integer Mapping

### Notes:

Specified Precision is what the user entered.

Adjusted Precision is rounded down to the nearest power of two (improved precision).

Actual Precision is computed precision after scaling factors have been computed from the length.

| IMAP Input Value | IMAP Output Value (Hex) | RIMAP Output       | Comments                                 |
|------------------|-------------------------|--------------------|------------------------------------------|
| 0.00000000       | 0x00000000              | 0.00000000         |                                          |
| 10.10000000      | 0x0A199999              | 10.09999996        |                                          |
| 20.20000000      | 0x14333333              | 20.19999999        |                                          |
| 30.30000000      | 0x1E4CCCCC              | 30.29999995        |                                          |
| 40.40000000      | 0x28666666              | 40.39999998        |                                          |
| 50.50000000      | 0x32800000              | 50.50000000        |                                          |
| 60.60000000      | 0x3C999999              | 60.59999996        |                                          |
| 70.70000000      | 0x46B33333              | 70.69999999        |                                          |
| 80.80000000      | 0x50CCCCCC              | 80.79999995        |                                          |
| 90.90000000      | 0x5AE66666              | 90.89999998        |                                          |
| 100.00000000     | 0x64000000              | 100.00000000       |                                          |
| NaN              | 0xD0000000              | NaN                | Quiet NaN                                |
| Infinity         | 0xC8000000              | Infinity           | Positive infinity                        |
| -Infinity        | 0xE8000000              | -Infinity          | Negative infinity                        |
| -1.00000000      | 0xE0000000              | IMAP_BELOW_MINIMUM | Below IMAP's minimum range (i.e., 0.0)   |
| 101.00000000     | 0xE1000000              | IMAP_ABOVE_MAXIMUM | Above IMAP's maximum range (i.e., 100.0) |

### 10.2.2 Test 2

| IMAP <sup>B</sup> (0.0, 100.0, 3)                                                                                                                                             |                         |              |          |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|--------------|----------|
| IMAP Details:<br>Min = 0.0<br>Max = 100.0<br>SF = 65536.0<br>SR = 1.52587890625E-5<br>Length = 3<br>Zoffset = 0.0<br>Actual Precision = 0.00001525878906250000 (1.525879e-05) |                         |              |          |
| IMAP Input Value                                                                                                                                                              | IMAP Output Value (Hex) | RIMAP Output | Comments |
| 0.00000                                                                                                                                                                       | 0x000000                | 0.00000      |          |
| 10.10000                                                                                                                                                                      | 0x0A1999                | 10.09999     |          |
| 20.20000                                                                                                                                                                      | 0x143333                | 20.20000     |          |
| 30.30000                                                                                                                                                                      | 0x1E4CCC                | 30.29999     |          |
| 40.40000                                                                                                                                                                      | 0x286666                | 40.39999     |          |
| 50.50000                                                                                                                                                                      | 0x328000                | 50.50000     |          |
| 60.60000                                                                                                                                                                      | 0x3C9999                | 60.59999     |          |
| 70.70000                                                                                                                                                                      | 0x46B333                | 70.70000     |          |

## ST 1201.5 Floating Point to Integer Mapping

|           |          |                    |                                          |
|-----------|----------|--------------------|------------------------------------------|
| 80.80000  | 0x50CCCC | 80.79999           |                                          |
| 90.90000  | 0x5AE666 | 90.89999           |                                          |
| 100.00000 | 0x640000 | 100.00000          |                                          |
| NaN       | 0xD00000 | NaN                | Quiet NaN                                |
| Infinity  | 0xC80000 | Infinity           | Positive infinity                        |
| -Infinity | 0xE80000 | -Infinity          | Negative infinity                        |
| -1.00000  | 0xE00000 | IMAP_BELOW_MINIMUM | Below IMAP's minimum range (i.e., 0.0)   |
| 101.00000 | 0xE10000 | IMAP_ABOVE_MAXIMUM | Above IMAP's maximum range (i.e., 100.0) |

### 10.2.3 Test 3

| IMAP <sup>B</sup> (-9.9, 110.0, 3)                                                                                                                                                            |                         |                    |                                          |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|--------------------|------------------------------------------|
| IMAP Details:<br>Min = -9.9<br>Max = 110.0<br>SF = 65536.0<br>SR = 1.52587890625E-5<br>Length = 3<br>Zoffset = 0.5999999999767169<br>Actual Precision = 0.00001525878906250000 (1.525879e-05) |                         |                    |                                          |
| IMAP Input Value                                                                                                                                                                              | IMAP Output Value (Hex) | RIMAP Output       | Comments                                 |
| -9.90000                                                                                                                                                                                      | 0x000000                | -9.90001           |                                          |
| 0.22500                                                                                                                                                                                       | 0x0A2000                | 0.22499            |                                          |
| 10.35000                                                                                                                                                                                      | 0x144000                | 10.34999           |                                          |
| 20.47500                                                                                                                                                                                      | 0x1E6000                | 20.47499           |                                          |
| 30.60000                                                                                                                                                                                      | 0x288000                | 30.59999           |                                          |
| 40.72500                                                                                                                                                                                      | 0x32A000                | 40.72499           |                                          |
| 50.85000                                                                                                                                                                                      | 0x3CC000                | 50.84999           |                                          |
| 60.97500                                                                                                                                                                                      | 0x46E000                | 60.97499           |                                          |
| 71.10000                                                                                                                                                                                      | 0x510000                | 71.09999           |                                          |
| 81.22500                                                                                                                                                                                      | 0x5B2000                | 81.22499           |                                          |
| 91.35000                                                                                                                                                                                      | 0x654000                | 91.34999           |                                          |
| 101.47500                                                                                                                                                                                     | 0x6F6000                | 101.47499          |                                          |
| 110.00000                                                                                                                                                                                     | 0x77E667                | 110.00000          |                                          |
| 0.00000                                                                                                                                                                                       | 0x09E667                | 0.00000            | Zero mapping                             |
| NaN                                                                                                                                                                                           | 0xD00000                | NaN                | Quiet NaN                                |
| Infinity                                                                                                                                                                                      | 0xC80000                | Infinity           | Positive infinity                        |
| -Infinity                                                                                                                                                                                     | 0xE80000                | -Infinity          | Negative infinity                        |
| -100.00000                                                                                                                                                                                    | 0xE00000                | IMAP_BELOW_MINIMUM | Below IMAP's minimum range (i.e., -9.9)  |
| 121.00000                                                                                                                                                                                     | 0xE10000                | IMAP_ABOVE_MAXIMUM | Above IMAP's maximum range (i.e., 110.0) |